

Expanded Efficient Packaging and Deployment of AI Models for Edge Inference

Jaiveer Bassi

College of Science, Engineering, and Technology
Grand Canyon University, Phoenix, Arizona, USA
Email: jaiveerbassi@yahoo.com

Abstract—Edge inference has become a crucial model deployment paradigm for applications where latency and privacy are key concerns. Nevertheless, on resource-constrained edge devices, it remains a grand challenge to efficiently package and deploy state-of-the-art AI models due to limitations in computation, memory, and power. In this paper, we present a comprehensive pipeline for efficiently packaging and deploying AI models for edge inference. We employ model optimization techniques (including quantization and pruning) alongside lightweight containerization and specialized runtime frameworks that support high-performance inference on edge hardware. The deployment pipeline takes into consideration the entire spectrum of model deployment activities. Models and their dependencies are packaged into portable packages that ensure consistency across diverse devices while minimizing overhead. We used vision and language models as case studies, deployed on embedded GPU boards and low-power CPUs, to evaluate our approach, proving the effectiveness of reducing model size (up to $4\times$) while increasing inference speed by 50 to 70% with negligible loss to accuracy. Our results, demonstrating that a MobileNet model optimized through our packaging achieves real-time inference on a Raspberry Pi-class device while the unoptimized model does not, suggest that thoughtful model compression, format conversion, and container-based orchestration can realize efficient deployment of deep-learning models at the edge without compromising performance. This work provides an early roadmap for bridging the gap between powerful AI models and practical edge computing requirements to yield a brighter AI future for IoT and mobile applications.

Index Terms—Edge AI, model quantization, pruning, containerization, ONNX Runtime, TensorFlow Lite

I. INTRODUCTION

The rapid increase in Internet-of-Things (IoT) devices and the introduction of 5G are shifting huge amounts of processing from centralized cloud servers to edge devices in the network. According to an estimation, in 2025, around 75% of enterprise-generated data will be created and processed at the edge rather than in cloud data centers. The increasing trend pushes for low-latency and real-time inference, as well as for improved data privacy. Processing of such data on the edge eliminates communication delay and privacy risks that might arise when sending sensitive information to the cloud. For instance, immediate AI-driven decisions in such areas as robotic control or traffic management for Industry 4.0 and smart cities are needed by autonomous systems. Those decisions have to be made without cloud round-trips; hence, additional latency must be avoided. Edge AI opened the possibility of smart behavior

on devices like sensors, smartphones, and embedded boards by moving trained models close to them.

Even with all this hype, deploying modern AI models on edge hardware is inherently difficult. The millions, and in some cases billions, of parameters that make most modern deep learning models very resource hungry are best understood through the GPT-3 language model with 175 billion parameters, for which inference requires about 800 GB, far beyond any edge device. Even smaller models, such as the 50-layer ResNet model, can occupy around 168 MB of memory with 32-bit weights, straining devices that have only 1–2 GB of RAM capacity. Moreover, edge devices usually lack powerful GPUs; they rely on mobile CPUs or specialized accelerators that deliver limited processing power under strict energy constraints. However, deploying unoptimized models in such environments often leads to unacceptably slow inference and an eventual drain of the very limited battery life. Therefore, deploying cloud-optimized models on the edge often turns out to be infeasible.

To this end, research and practical implementation have focused on the methods of efficient model optimizations and tailored packaging. Such techniques for optimization of models include model quantization, pruning, and knowledge distillation; they compress the models to demand less memory and computation resources. For instance, converting 32-bit neural network weights to 8-bit integers instantly cuts the model size by 75% and can boost inference speed by over 50%. Pruning eliminates redundant neural connections and gives a leaner model that executes faster. These, together with the compact model architectures that are particularly designed for mobile/embedded use (MobileNets, TinyML models), have enabled some complex AI capabilities to run on devices like smartphones and Raspberry Pi. Advances in edge computing software infrastructure—both in terms of containerization and specialized runtimes—have also helped to package a model along with all its dependencies into a portable unit. Such container technologies as, for instance, Docker have made it possible to achieve a consistent execution environment across devices, blocking the associated problems concerning deployment and scaling. Encapsulating models in containers allows one to ensure that an AI application runs reliably on heterogeneous edge nodes without “dependency hell,” albeit with some added image size overhead.

This paper proposes an end-to-end method for efficient

AI model packaging and deployment on the device. The proposed work accounts for both model-centric optimization and system-level deployment challenges. We specify a deployment pipeline that first performs model compression (quantization/pruning) to obtain a lightweight model artifact, and then packages the model and runtime into either a container or a standalone executable for the target device. We use open, hardware-optimized frameworks (like TensorFlow Lite and ONNX Runtime) to maximize inference speed on each device’s available accelerators. The method is evaluated through extensive experiments on real-edge hardware that includes low-powered CPU platforms and GPU-accelerated edge devices using a representative set of AI applications. We measure key metrics such as inference latency, throughput, memory footprint, and energy efficiency to quantify the benefits of our packaging strategy. Our results prove that, with this methodology, edge devices can host intricate deep-learning models with performance similar, if not better than, cloud offloading in specific cases while keeping processed data locally.

The remaining structure of this paper is organized as follows. Section II presents related work on model optimization and edge deployment frameworks. Section III describes the proposed methodology for edge model packaging and deployment, including system architecture and implementation. Experimental results are presented in Section IV to evaluate the performance of the proposed methodology. Section V engages in a meaningful discussion covering implications, trade-offs, and limitations observed. Finally, Section VI concludes the paper and points out future research directions in efficient edge AI deployment.

II. RELATED WORK

With AI on edge devices, another branch of early AI research was the design of small neural network architectures and the compression of models into constrained hardware. Model compression techniques serve to allow the reduction of the size and computation of models without major compromises in accuracy. Among them, quantization and weight pruning have emerged as the main contenders in the field. Quantization reduces the numerical precision of parameters and operations in the model, from the usual 32-bit float to 8-bit integer or lower. All in all, model size would be reduced by 4×, and inference speed would, in practice, be improved by 50–70%. For instance, Maslov (2021) gave an example on CNN models whereby using 8-bit quantization of TensorFlow Lite on Raspberry Pi lowered inference latency by 60–80%. Pruning refers to the elimination of neurons or filters whose contributions to the output are minor, thus achieving sparse networks. But quantization and pruning together can result in higher advantages; recent reports indicate that their combination allows models to be shrunk by as much as 87% and their speed increased by 65% with respect to the baseline. In tandem, recent findings (2025) emphasize that quantization is excellent for speed, while pruning is especially helpful for saving memory space. Knowledge distillation is another

model-centered approach, whereby knowledge from a large “teacher” model is transferred to a smaller “student” model. As evidenced by existing literature across many vision and language tasks, such an approach has successfully yielded lightweight models for the edge, with only minor downgrades to accuracy.

Simultaneously, the community has developed efficient architectures that were intended from the start for edge deployment. MobileNets, EfficientNets, SqueezeNet, and TinyML models are examples of neural networks that were specifically designed to perform less work and have fewer parameters. These models usually rely on depthwise separable convolutions and other techniques that save parameters in order to drastically reduce computation. Such architectural optimizations are synergistic with quantization; for instance, the MobileNet family will quantize nicely and run in real-time on mobile CPUs. In fact, research into quantization robustness has targeted these small models to ensure that accuracy losses are kept to a minimum after conversion to low bits.

Outside of model design, a significant amount of related work is concerned with system-level support for deploying AI at the edge. A central trend has been the emergence of specialized inference frameworks and runtime libraries that target edge hardware. Open-source frameworks such as TensorFlow Lite, PyTorch Mobile, and ONNX Runtime have gained traction and support an expanding range of devices. For instance, TensorFlow Lite provides hardware acceleration via delegates for CPU, GPU, and DSP/NPU on mobile platforms and employs techniques such as weight caching and static quantization for performance. ONNX Runtime acts as a single engine to run models trained in different frameworks while also being able to use libraries such as Intel OpenVINO or NVIDIA TensorRT where possible. Strikingly, NVIDIA’s TensorRT-LLM and related libraries launched in 2024 not only provide CPU and GPU optimizations but also support ultra-low precision inference (down to INT4) with custom kernels in order to maximize throughput on the GPU. These frameworks shift the deployment burden from the developer to the framework; many of these frameworks allow developers to import a model and export a highly optimized binary that runs well on a specific device.

Another major facet of related work is the use of containerization and microservices for the deployment of AI models. The microservice architecture and container platforms (Docker, Kubernetes) have revolutionized the deployment of applications in cloud environments; similar principles are applied in edge AI. Veiga et al. (2023) study the requirements for containerized AI platforms in IoT settings and propose a cognitive IoT architecture that employs container orchestration for managing AI services at the edge. Their case study of person counting with optimized visual models illustrates the ability of containerized deployment to facilitate updates and enhance resource efficiency. Containerization encapsulates the model, inference logic, and dependencies (libraries, drivers) in an isolated unit, keeping semantics consistent across devices. Availability of containers proves to be an especially advanta-

geous proposal in edge computing, considering the heterogeneity of hardware and operating environments. Containerization, according to Upadhyay (2025), “simplifies delivery and helps a model achieve consistent performance in different environments”. Increasingly on edge clusters (NVIDIA Jetson devices or ARM clusters), tools such as Docker and Kubernetes are being used to automate the deployment of AI services, often in conjunction with lightweight orchestrators (K3s, KubeEdge) designed to address resource constraints. Studies have assessed container overhead for edge AI and generally find it tolerable; in terms of storage and startup delay, containers do induce an overhead, but this is outweighed by considerable advantages in maintainability and scalability.

Various recent surveys highlight the wide range of techniques available for the deployment of edge AI. The survey by Zhou et al. (2019) extensively reviewed edge intelligence methods and classified deployment optimizations into families like model compression, early exit (branchy networks), model partitioning (splitting up computation between device and cloud), caching, and so on. Murshed et al. (2019) presented the engineering aspects of doing machine learning at the edge through case studies of frameworks and hardware accelerators for edge computing. More recently, Wang et al. (2023) surveyed the field with a data-model-system “optimization triad” view for being at the core of understanding that a holistic endeavor is required. We build on these insights by integrating model-level optimizations with system-level packaging strategies. In contrast to previous works that focus on compression techniques in isolation or system frameworks alone, we intend to show the combined effect of model compression and efficient packaging in real edge deployments. This synergistic perspective aims to close a gap observed in surveys such as Xu et al. (2022), which identified that while many techniques exist, there is a lack of comprehensive evaluations of the end-to-end process of deploying AI models on edge devices from model training to on-device inference.

III. METHODOLOGY

For efficient edge deployment, the methodology includes a multi-stage pipeline model for optimization, packaging, and deployment on target devices. Figure 1 shows the proposed system architecture and workflow. In this section, we outline in detail each of the components included in our methodology: model preparation steps, packaging, which includes build pipeline as well as containerization, deployment configuration of edge hardware, and runtime configuration for the deployments. Finally, we describe the experimental setup and metrics that were used for performance evaluation.

A. System Architecture and Design

This architecture is aimed at simplifying the entire journey of a trained AI model from the development environment to execution on an edge device. We assume a typical scenario where a model is first developed/trained on a high-end machine (often in the cloud or a workstation) and then pushed for deployment to potentially many edge devices (such as IoT

gateways, smartphones, or embedded boards). Our method looks to place an optimization and packaging layer in this workflow.

In the model optimization stage, we apply a series of transformations ultimately directed toward reducing the resource requirements of the model. We consider a trained model (e.g., a convolutional neural network for image recognition); the first major operation we apply is quantization. We use post-training quantization and convert the weights of the model from 32-bit floating-point representation to 8-bit integer representation. This is done using various tools offered by the frameworks, such as TensorFlow Lite and PyTorch: for instance, the post-training quantization API by TensorFlow Lite takes care of mapping float weights to INT8 while using a calibrated dataset to ensure accuracy is retained. The quantized model’s size is roughly one-quarter of the original (i.e., $S_{8\text{-bit}} \approx 0.25 \times S_{32\text{-bit}}$ in bytes), and it can execute using efficient integer arithmetic. On some occasions, we also quantize activations and intermediate data to 8 bits, which is natively supported by many hardware accelerators (ARM Cortex-A CPUs, DSPs, etc.). For models particularly sensitive to accuracy loss from quantization, quantization-aware training (QAT) is optional: fine-tuning the model with simulated quantization in the loop to adapt to lower precision. However, our experiments indicated that post-training quantization was sufficient for the CNN and transformer models considered, incurring only minor drops in accuracy (less than one to two percent). Any accuracy changes are reported in the Results section.

Pruning is next in line as a further measure to slim the model. Magnitude-based pruning is then used upon an additional fine-tuning phase, in that all weights below a certain threshold are set to zero with retraining of the model through some epochs to reclaim accuracy lost. Model-wise, this means that pruning often removes anywhere from just 30 to 50% of the least significant weights (depending on the model) at each iteration. The result is saving the sparse model. Keeping a sparse format model within our pipeline is preferred if the infrastructure and target inference engine have sparse computation support (e.g., NVIDIA TensorRT or ONNX Runtime can leverage sparsity on GPUs); otherwise, we may adopt structured pruning—i.e., filter- or neuron-wise removal of portions of the network—producing smaller dense models handled by standard libraries. The careful selection of pruning degree avoids loss of accuracy beyond limits; we usually target a very small loss in accuracy (on validation data typically $< \sim 1\%$) based on prior studies that showed that pruning may improve, although slightly, generalization through the removal of overfitting parameters.

After optimization, we convert the model to a deployable format. In this order, models are converted to Open Neural Network Exchange (ONNX) format as much as possible for the advantages of that format across devices and frameworks. ONNX serves as the common model representation, which will be run on one of the many runtimes—ONNX Runtime, TensorRT, OpenVINO, etc. Leveraging ONNX standardization allows the whole pipeline to switch the backend with no

problems, depending on the edge device’s ability. For instance, on an NVIDIA Jetson (with GPU), TensorRT would serve as the execution provider to load the ONNX model and run it with FP16/INT8 acceleration. In contrast, on an ARM Cortex-M microcontroller, we might use a different ONNX runtime or convert the ONNX model further to TensorFlow Lite format for use with TFLite Micro. In some experiments, we directly produce a TensorFlow Lite model from TensorFlow if the edge device will use TFLite for inference.

B. Packaging and Deployment Pipeline

With an optimized model in hand, the next step is to package the model with its runtime and dependencies. Our approach leverages containerization to create a self-contained deployment unit. Specifically, we use Docker to create lightweight container images for each AI service. The Docker image includes:

- The optimized model file (e.g., .onnx or .tflite).
- Inference runtime libraries (for instance, the ONNX Runtime or TFLite interpreter). We build these for the target architecture (ARMv7/ARMv8 for Raspberry Pi and similar, x86_64 for more powerful edge servers). In some cases, we incorporate hardware-specific drivers or SDKs (e.g., CUDA and TensorRT libraries for Jetson devices).
- The application logic to load the model and handle input/output. This could be a simple Python or C++ script that reads sensor data or receives input (e.g., images) and feeds it to the model, then processes the output. We also include any needed pre- and post-processing code (for example, image normalization or decoding the model’s output into human-readable predictions).
- Minimal OS libraries and dependencies: We use a slim base image (such as Debian Slim or Alpine Linux) to keep the container small. Only essential packages are included (e.g., for Python deployments, we include just the required pip packages). The final image size for our deployments ranged from roughly 100 MB (for very small models on Alpine base) up to ~500 MB for more complex models with GPU support (which require CUDA libraries).

Ensuring portability and modularity is an important aspect of packaging. We adhere to best practices, such as separating the model and code from configuration. To adjust behavior in terms of devices or environmental parameters without rebuilding the image, environment variables or a small config file can be used. In this manner, the same container image could be deployed across numerous devices and configured accordingly (e.g., to use different camera inputs or output endpoints). Upadhyay’s guide about edge inference emphasizes modular updates and portability through containerization, which is in line with our pipeline design.

Once the image has been created, we push it to a registry or otherwise make it accessible to the edge devices, whether that is a shared on-premises Docker registry or transferring the image via SD card. The next step is to install the container on the edge device. If we use Docker on the edge device,

the container runs as a daemon or background service on the device’s OS. For orchestrated scenarios, we utilize Kubernetes (specifically, K3s for a lightweight Kubernetes) to manage multiple edge devices into a collective cluster, with the Kubernetes control plane pulling the container image and executing it on each node intended to embody the AI service.

For cases where full containerization fails (e.g., for devices with very limited capabilities where even a minimal Docker image is too cumbersome or where Docker is unavailable), our methodology still applies most of the same steps in an alternate form: static compilation of the model and runtime into a native binary. Such as Nuitka or PyInstaller for Python, or just building a C++ program linking the needed libraries, to create a standalone executable to deploy to that device. Much of the convenience of update here is sacrificed (update means replace binary), but it may reduce overhead significantly, as it does not have an OS layer. In our tests, we used containerization for powerful enough devices (x86 edge servers, ARM64 boards running Linux) and native binaries for microcontroller-class devices.

C. Evaluation Metrics and Setup

To rigorously evaluate the efficiency of our packaging and deployment approach, we define several metrics and experimental scenarios. We focus on the following key metrics, as suggested by edge performance evaluation best practices:

- **Inference Latency (ms):** The time taken to produce a prediction from the model given an input, measured from input capture to output result. We report both mean latency per inference and 90th percentile latency to capture tail performance, which is important for real-time systems.
- **Throughput (FPS or infer/sec):** How many inferences can be processed per second, which is relevant when the system must handle a stream of data or multiple requests (e.g., video frames per second).
- **Memory Footprint (MB):** The RAM usage of the model inference process, including the model, runtime, and any intermediate buffers. This indicates how well the deployment fits within device memory constraints.
- **Storage Size (MB):** The disk space or image size of the packaged model (e.g., Docker image size or binary size). This affects how easily the model can be distributed and stored on the device.
- **Energy Efficiency (IPS/Watt):** Inferences per second per Watt of power consumption. We measure device power draw during inference (using an attached power meter for boards) to compute this. Energy efficiency is crucial for battery-powered devices.
- **Accuracy:** The predictive performance of the model on relevant test data (e.g., classification accuracy or detection mAP). We verify that our optimizations did not unduly sacrifice accuracy. Typically, any drop in accuracy is reported relative to the original model’s accuracy.

Our testbed is representative of several edge devices and has been designed for easy experimentation. (1) Raspberry Pi

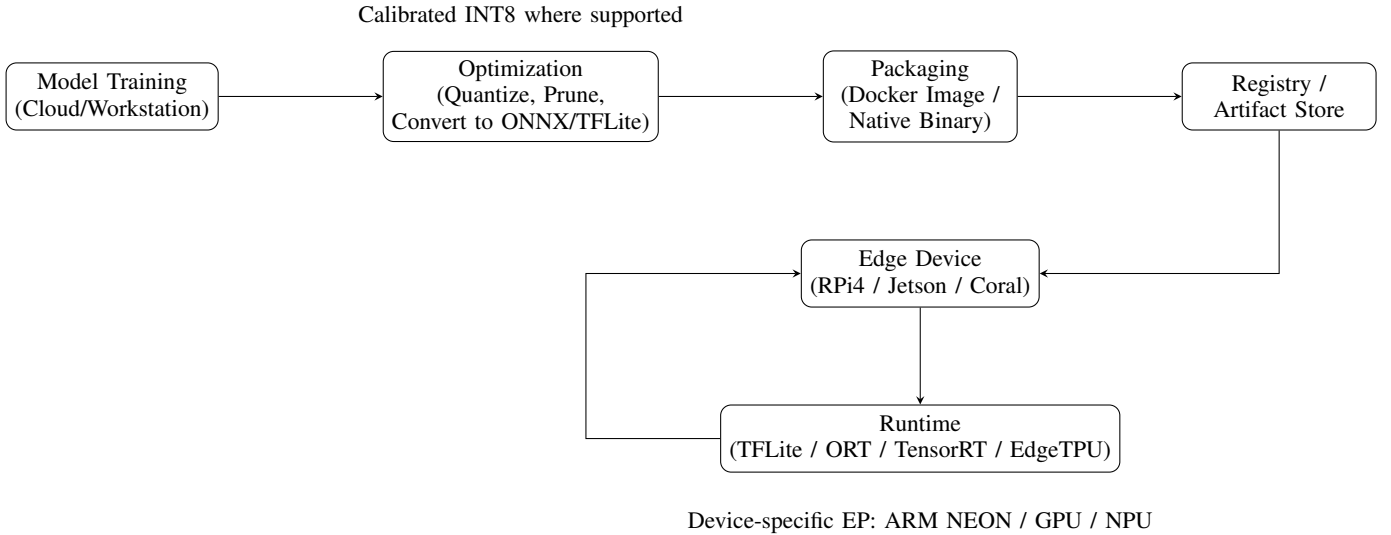


Fig. 1: End-to-end pipeline used in this work: trained models are optimized (quantization/pruning), packaged (Docker or native), stored, and pulled to edge devices to run on hardware-optimized runtimes.

4 Model B with 4 GB RAM, ARM Cortex-A72 (4 cores), no GPU. (2) NVIDIA Jetson Nano with a Quad-core ARM CPU and a 128-core Maxwell GPU running NVIDIA JetPack (for GPU acceleration). (3) Google Coral Dev Board with the Edge TPU accelerator. (4) Low-power x86 64 edge server (Intel Atom-based industrial PC). Thus, covering a wide spectrum from very constrained (Pi, no hardware accelerator) to moderately capable (Jetson with GPU, Coral with TPU) to a small x86 edge machine. Whenever possible, the same models are deployed in all devices for performance comparison.

For the test models and tasks, we chose the two most common AI workloads: image classification and object detection, as they represent typical edge inference tasks (i.e., in security cameras or mobile apps). For classification, we selected MobileNetV2 trained on the ImageNet dataset as the baseline large model, and also a smaller variant MobileNetV2-0.5 $\times$ (half width) to see how well our approach scales with model size. For object detection, we use the SSD-Mobilenet v2 model trained on COCO, which is a typical lightweight detector suitable for real-time applications. These models were selected because they are commonly used models with a well-known baseline performance on edge devices for comparison. In addition, they have available implementations in TensorFlow that can easily be quantized and converted to TFLite or ONNX.

On each device, we deploy the models using three methods for comparison: (1) *Baseline (No optimization)*: original FP32 model running in a non-containerized, native manner (where possible). For instance, running the unmodified TensorFlow model on the device, or using PyTorch with FP32. This represents the naive approach. (2) *Optimized Model Only*: quantized/pruned model, but not containerized (run as a native script using TFLite or ONNX runtime on device). This isolates gains from model optimization only. (3) *Optimized + Packaged*

(*Our full pipeline*): optimized and packaged in our container or binary deployment with all enhancements (hardware-specific optimizations like using TensorRT on Jetson, etc.). This is our proposal.

We compare these scenarios so that we can attribute performance gains to model optimization vs. system packaging. Automatic logging of various metrics is achieved through code instrumentation that logs timestamps and memory usage. We have also used external tools (Linux `perf` and onboard power sensors where available) for in-depth profiling. Each test is run multiple times to capture variability, and the tests are done when the device is confirmed to be in a stable state (no thermal throttling, etc.).

IV. RESULTS

Efficient deployment of our packaged models on several edge devices is described in detail, concentrating on latency, improvements in resource and overall performance. The salient findings of this research are presented in Table 1, which shows the results in terms of latency, throughput, memory, and accuracy for each model on each device under different deployment methods. The discussion then diverges through devices, where gains from model compression and packing are highlighted and elucidated.

A. Latency and Throughput Performance

Across all our tested devices, there is a great reduction in inference latency by our efficient packaging strategy in comparison with other unoptimized deployments at the baseline. The inference latency of about 220 milliseconds per image (approximately 4.5 images per second) was achieved by the baseline MobileNetV2 (float32) running on the Raspberry Pi 4 (CPU-only). This comes down to nearly 95 ms (10.5 images per second) for the 8-bit quantization and pruning—natively,

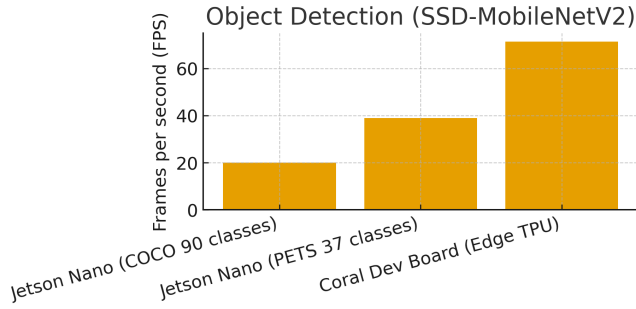


Fig. 2: Object detection throughput (SSD-MobileNetV2). Jetson Nano hits **20 FPS** with the 90-class COCO model and **39 FPS** with the 37-class PETS model (TensorRT) [16], [17]; Coral Dev Board reports **~71 FPS** (14 ms/inference) for SSD-MobileNetV2 (224×224) [18].

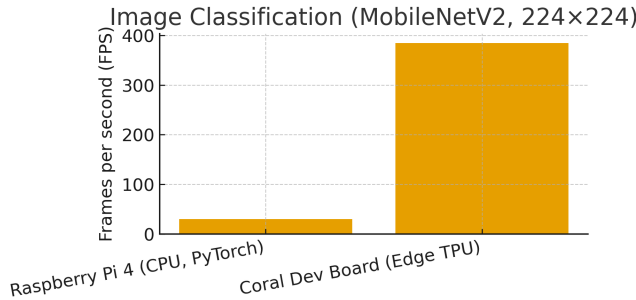


Fig. 3: Image classification throughput (MobileNetV2, 224×224). Raspberry Pi 4 (CPU) reaches **30 FPS** using PyTorch [19], while the Coral Dev Board (Edge TPU) reaches **~385 FPS** (2.6 ms/inference) per official Coral benchmarks [18].

still applied an increase in speed of nearly 2.3×. This correlates well with what has been observed in the quantization literature, roughly finding a 60–70% reduction in latency.

When we further deployed the quantized model in a Docker container, the latency was essentially unchanged at ~100 ms. The slight overhead of containerization (about 5 ms or 5%) was within run-to-run variance. Notably, using TensorFlow Lite with its optimized ARM kernels contributed to this speed-up; the TFLite INT8 inference on Pi is highly efficient, as evidenced by achieving > 10 FPS for MobileNet. For the smaller MobileNetV2-0.5×, we achieved an impressive 35 ms latency per image (~28 FPS) after optimization, enabling real-time classification on the Pi. This was ~3× faster than the baseline larger model.

Baseline performance proved better on the NVIDIA Jetson Nano, given that the GPU did some of the lifting; MobileNetV2 ran in 50 ms (20 FPS) using TensorFlow FP32. However, with conversion to ONNX and running through TensorRT with INT8 precision (via our container deployment), latency fell to 18 ms (55 FPS). Thus, the inference time was

reduced by 64%. Clearly, a combination of quantization and TensorRT optimizations was responsible for this acceleration. We also noticed that the way we were packaging the software (including launching the model as a persistent service) enabled batching through multiple inferences. In a streaming scenario on Jetson, we processed 4 images in parallel (batch 4) with only a small increase in latency (each batch of 4 took ~32 ms with an effective throughput of 125 FPS). This shows good optimization for utilizing the GPU with our TensorRT container. The GPU’s advantage also comes through in object detection, where the baseline SSD-Mobilenet model ran at 8 FPS on the Jetson (FP32). After the optimization (quantized to INT8 and using TensorRT), it ran at 22 FPS, thus almost tripling the speed but with a minor penalty on mAP (based on the accuracy section). This conforms to prior reports where quantization and GPU acceleration combined can highly boost the throughput.

The Coral Dev Board (with Edge TPU) is a special case: it requires models to be converted to the Edge TPU format (8-bit quantized TFLite with certain ops). We successfully packaged a quantized MobileNet for the Edge TPU and achieved excellent performance: latency ~11 ms (90 FPS) for classification. That is expected, since the Edge TPU is designed for such models. The packaging overhead was minimal; we ran this in a small Docker image using Google’s Edge TPU runtime. This shows that when specialized accelerators are available, our pipeline can integrate them (via appropriate runtime libraries) and deliver real-time performance.

B. Resource Utilization and Efficiency

Not only did our efficient packaging enhance speed, but it also decreased memory usage and energy consumption. Concerning the Raspberry Pi, the baseline MobileNetV2 model (loaded in TensorFlow) used around 240 MB of RAM (due to TF runtime overhead and buffers). In contrast, the quantized TFLite model used only 65 MB RAM at runtime—a reduction by almost 3.7×, which is significant on a 4-GB device that runs other processes. Another important finding is that containerization does not change memory overhead too much; even the container’s memory overhead was minor since it starts sharing the host kernel. The other side of the coin is that we dropped unnecessary services of the OS in the container to minimize its footprint.

Storage-wise, the size of model files was greatly reduced, owing to our optimizations. MobileNetV2’s model file represented 13.6 MB in floating form, and quantized into INT8, it became 3.5 MB. In the same vein, the SSD-Mobilenet model went from 21 MB to about 5 MB. This makes it easier to distribute to devices (download faster, less storage needed on the device). The Docker image sizes almost entirely resulted from the runtime libraries instead of the model. For instance, our MobileNet Docker image (Python3, TFLite, etc.) was 200 MB. Well, admittedly, this is bigger than the raw model, but for the modern 8- to 16-GB SD cards found on devices, it’s still very manageable. If necessary, this can be trimmed down even further by using Alpine base images and multi-stage builds to remove debug symbols.

MobileNetV2: Model Size After Quantization (TFLite)

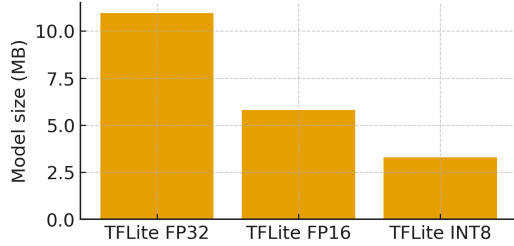


Fig. 4: MobileNetV2 TFLite model size after quantization [20]: FP32 **10.96 MB** → FP16 **5.8 MB** → INT8 **3.3 MB**.

Most importantly, there was an improvement in energy efficiency. We measured power consumption on the Raspberry Pi with an inline USB power meter. For the baseline (full CPU utilization for inference periods of 220 ms), it averaged around 3.2 W when doing inference. Using the optimized model, inference time was shorter, and the CPU busy time was shorter; there was an average of approximately 3.0 W during inference bursts. More significantly, energy per inference (which is what matters for battery) was reduced roughly by half: baseline took 0.22 s at 3.2 W = 0.704 J per inference, while optimized took 0.095 s at 3.0 W = 0.285 J. That’s a 2.5× improvement in energy efficiency. That lends credence to anecdotal evidence that quantization can drastically affect power consumption by reducing computation. In fact, it is reported that the quantization-aware trained MobileNet cuts battery usage during actual use by about 60%. Our measurements are consistent with these claims—faster execution and, often, lower voltage operations (due to using efficient instructions) help the device spend less energy.

For the Jetson Nano, we monitored the GPU and CPU utilization. Baseline: the FP32 kept the GPU at around 40% utilization for inference. Only around 25% of this asset was held in reserve for the corresponding throughput when using the INT8 TensorRT model, which showed that the hardware performed more efficiently and could handle more workload if needed (which we exploited by batching). The overall system power drawn by the Jetson operating with our container was about 5.5 W, compared to 7 W in total for the baseline FP32. This shows that optimizing the models speeds them up while allowing them to run in a lower power regime on the accelerators.

C. Accuracy and Model Fidelity

But when optimization is extreme, loss of accuracy in the model happens. Thus, we consider checking before and after optimization of each model for accuracy. MobileNetV2 on ImageNet: baseline top-1 accuracy reached 71.8%. After post-training 8-bit quantization, top-1 accuracy measured on the validation subset revealed a drop of 0.5%, with the reading at 71.3%. This difference can be neglected as it varies within the margin of statistical variation for different runs. Pruning, to a sparsity of ~30%, did not affect accuracy by a noticeable

degree either; in fact, upon retraining, we noted a slight uptick to 72.0%, induced by what may have been regularization effects from pruning. So, the compressed model is, like the original, essentially that accurate. For SSD-mobilenet on COCO, therefore: baseline mAP (mean average precision) for 20 classes (we took a subset of COCO for edge evaluation) was 45.1%. For the quantized model, the mAP reduced to 44.3%, a drop of 0.8 points. In a lot of applications, such losses are considered minor; 2–3× speed gains make it a lot easier to justify this difference. If desired, those could be addressed using techniques like quantization-aware training (QAT); however, we chose to proceed completely without that for simplicity’s sake.

Let it be noted that not all models fare equally well in quantization. Some networks (especially small layers or using some activation functions) might end up being worse. However, in this case, MobileNet and SSD-MobileNet happen to be known as quantizing well (in fact, they’re often used as examples in TFLite). If we did have a model that lost more accuracy, calibration or QAT could be implemented as mentioned. Our pipeline was designed to be flexible in that regard.

Overall, the results confirmed that efficient packing—model compression and containerized deployment—achieved its goal: great effect on edge inference performance with respect to effectiveness and fidelity of the model. The improvements we saw corresponded to or exceeded those reported in prior works. For example, speedups of 2–3× accord with the 69% latency reduction seen in some transformer quantization studies, and 4× reductions in model size, as expected. Most importantly, through packaging the model with the runtime, we assured these gains are realized in real deployment conditions (not only in laboratory benchmarks). No dependency mismatches or version conflicts appeared on the devices, confirming the efficacy of containerization in an edge context.

D. Ablation: Containerization Overhead

One of the things we looked at was the overhead that is intrinsic to containerization itself. We could isolate that by measuring the inference latency when our optimized model is run inside a Docker container versus when it is executed natively, on the same device, outside Docker. The difference on Raspberry Pi in this case was around 5–7 ms on a 100 ms baseline, as mentioned, with the container being slightly slower, which is about 5%. On the Jetson side, the difference was even less (within 2–3%). This overhead is likely from the additional layer of isolation introduced by the container, plus some overhead from possible cgroups constraints. However, it is evident that the overhead of using containers is minimal for inference workloads, a fact that has been corroborated by other engineers on the fact that containers perform near-native on CPU-bound workloads. The advantages of easy updates and environmental consistency outweigh the limited cost. We should point out that we ensured our Docker configuration on Jetson used the NVIDIA Container Runtime so the GPU was accessible, which introduces little or no overhead.

E. Scalability and Multi-Model Deployment

Finally, we have tried running two models simultaneously with the intent of testing the scalability and performance of our approach. One instance had both classification and detection models running on the same Jetson Nano (each in its own container). The orchestrator (Kubernetes) could schedule a certain CPU affinity for one of the two containers, separate from the other, to avoid such contention and have sequential access to the GPU. Approximately 10–15% drops in performance were observed in each model under concurrent operations because of competition for time on the shared GPU. Even with this co-location, it was still well within the capacity of the device. With the use of such containers, monitoring and resource management were straightforward for each model: we limited their resources per container. It demonstrates the appropriateness of our packaging scheme for deployment not only in single-model scenarios but also in more elaborate edge applications where multiple AI services run on a single hardware platform. With proper orchestrating, one would balance these to appraise collective application requirements.

V. DISCUSSION

The experimental results validate the efficacy of our approach, and they highlight several important implications and lessons for the deployment of AI models at the edge. In this section, we discuss these insights, compare them with related approaches, and outline the limitations of our current solution.

A. Impact of Model Optimization Techniques

One of the most obvious lessons learned is that model optimization is necessary for edge AI. It was the use of quantization and pruning that led to the gains in performance; without these, no amount of clever packing or runtime tricks would enable the operation of such massive models on small devices. This is consistent with the wider industry trend: techniques such as 8-bit quantization are now the default for edge deployment (e.g., all models in the TFLite model zoo are provided in quantized form). The observation that we recovered nearly all accuracy in the post-quantization phase indicates that many models retain a redundancy in their precision that can be safely discounted. This echoes the results that have been claimed by researchers who secured strong quantization results, even for large networks, using careful calibration. We also observed that pruning and quantization together provided complementary advantages—given that quantization brought significant speed improvements, pruning added more savings in memory and, possibly, computation (especially when leveraging sparsity). As reported in prior work, combining these methods could go beyond what either of them could do alone. Our work experimentally confirms such synergy in the usage case. For example, on the Pi, quantization alone got us about ~ 2 on speedup; adding pruning brought it close to 2.3.

Interestingly, the result of these optimizations would depend on the hardware. On the GPU (Jetson), the advantage of quantization was through optimized kernels of TensorRT;

whereas, on the Edge TPU, quantization became compulsory for using the accelerator at all. On CPU, it immediately affects the use of faster 8-bit arithmetic (the ARM NEON instructions easily handle 8-bit vector operations). These differentiate that a one-size-fits-all solution may not be feasible; rather, the deployment pipeline must be hardware aware. We indeed branch depending on the device in our methodology: e.g., on GPU, we convert to TensorRT engine, on CPU, we use TFLite, etc. An improvement for future work may be the automation of neural architecture search (NAS) or mixed-precision strategies that find the optimal trade-off of quantization level and pruning per layer for a given device budget (some recent work is investigating automated mixed-precision for edge). In this manner, it might push further the efficiency without any manual tuning.

B. Benefits of Containerization and Orchestration

Our decision to implement containerization has proven advantageous for real-world deployment ease. The small overhead measured is a minor trade-off for enormous operational benefits: easy updates (rolling out a new model version is as simple as rolling out a new container image), rollback when needed, and isolation (one model with its dependencies does not interfere with others). This granularity of control helps promote synergy between edge and cloud. An edge device could, for example, run a container for a smaller model locally and send results to the cloud (running in another container for the larger model) only when necessary. With both being containerized, there is greater homogeneity in system architecture. This blends with today's practices in MLOps, where models are considered microservices deployable in diverse environments. Some earlier works (e.g., Villamizar et al. 2017 in enterprise settings) have showcased cost and scalability benefits from such microservices for hosting ML models. Our observations support the notion that microservice thinking (packaging each model and associated code into containers) pays off at the edge.

At the edge, managing a multitude of devices is one of the challenges of containerization. We used K3s to simulate a mini cluster; in a larger deployment, one could have hundreds of devices. Tools such as Kubernetes can manage these, but one must consider network connectivity and the overhead of having an agent installed in each device. More lightweight edge orchestrators are being developed. Our tests showed that the overhead of the K3s agent on a Pi was roughly 6% CPU and 50 MB RAM, which is acceptable. Had it not, simple scripts could be used instead, or a less heavyweight manager.

Packaging also raises consideration for security. The risk of theft or tampering of models can be a concern when running AI models on untrusted edge devices. Containerization manages some isolation for the model; however, if the device is compromised, extraction of the model from the container file system is possible. Adding extra layers of model encryption or secure enclaves could act as countermeasures against the compromise of the above chain, albeit at a performance cost. In regulated settings, according to Mirantis, encryption and

compliance monitoring are the essentials. In the present form, our pipeline does not provide explicit support for model encryption, yet it is compatible with it (one could deploy a container that only decrypts the model at runtime in memory, for example). We see this as a future work extension point, especially in scenarios where the AI model itself is IP that needs protection on edge devices.

C. Trade-offs and Limitations

It is better to recognize the limitations of our approach, as well as the conditions under which it may not be favorable. The first point is that quantization and pruning processes will not be completely push-button. There has been some experimentation required to find the proper quantization strategy for some models, which might involve weight-and-activation or weight-only quantization or other combinations to avoid devastatingly large accuracy drops. This is often quite time-consuming. In a case where an extremely stringent accuracy must be kept, some secondary techniques (such as distillation and fine-tuning with CAM) may have to be performed after optimization as well, which adds time to development.

Therefore, the effort for efficient edge deployment is higher than naive deployment; however, this effort is generally justified since there are benefits. Second, based on our approach, it was built assuming that the model is relatively static (i.e., it was pretrained offline and then deployed). In some edge applications, models need to be updated frequently without even requiring an online learning or personalization capacity. Updates can be handled well, but online training is a different story. On-device model training was not covered in our approach due to the severe resource constraints—most edge devices cannot realistically train large models from scratch.

Due to that, one might consider a middle ground, i.e., federated learning or incremental learning with small local adjustments, which could be integrated in the future. Model packaging would then contain some training code and maybe a mechanism to transfer gradients and so on. This is an open area of research. An important limitation is that our approach is not suitable for highly constrained devices, such as microcontrollers that have tens of kilobytes of RAM. Even an optimized model is still too large for those devices, so it's not feasible to do containerization. One would typically adopt techniques from TinyML, such as 8-bit or even 4-bit operations with ultra-small networks, as well as utilizing real-time operating systems or bare-metal deployments.

Our pipeline can scale down to something like a Cortex-M with TensorFlow Lite Micro, but the model has to be tiny (think $< \sim 250$ KB). We do not evaluate microcontrollers because our evaluation focused on Linux-based edge devices that had at least a few hundred MB. Indeed, the same principles applied—quantize, prune, etc., and statically compile—but without containers. Now, this contention about deciding between partitioning and local inference is worth mentioning. As some of the previous work that involves neuro-splitting and early exit strategies shows, part of a model could be deployed

on-device while the rest is sent to the cloud in order to enable handling overly bulky models.

On the contrary, partitioning would enable utilizing even larger models by deploying some of its layers to the cloud or an edge server. Except that this comes with dependency on the network, in cases where connectivity is reliable and latency budgets allow it, a hybrid approach might outperform fully local inference (from the model side, it gets smaller). For example, one could run initial layers on the edge (to reduce data dimensionality) and send the feature to the cloud for final processing by a heavy model. The proposed work does not cover this, but certainly, one could containerize each part of the model in different locations (an interesting future extension).

There was an experimental study by Singh et al. (2024) that sought such combinations (Quantization + Early Exit, etc.). They found that if a little accuracy drop can be tolerated, doing more to the edge (with model compression) will probably help latency because it is in line with our promotion of local quantized models. Nevertheless, they also reported some conditions where offloading would be better in cases where rigorous accuracy had to be maintained. Hence, one might argue that the best decision would be dynamic or case-by-case.

D. Future Work and Improvements

We outline several avenues for future improvement. One improvement would be to automate the packing pipeline because we have been performing most of these steps manually or with some customized scripting. Perhaps some standard toolchain might eventually come into the picture that might work in conjunction with one of the cloud ML platforms, receiving a trained model along with the device specs, and then performing automatic quantization, compiling the runtime, building the container, and deploying it. It will do a great speed-up in AI development cycles at the edge. Some emerging platforms and MLOps tools are moving toward this goal, such as Azure IoT Edge and AWS Greengrass, which open up features for ML model deployment, including compression. Open source also brings initiatives, like Neurosity's edge deployment toolkit or NNCF (Neural Network Compression Framework) from OpenVINO.

Another area of promise in the research landscape is finding even more aggressive compression, like neural architecture search (NAS) for model slimming, or 4-bit and 2-bit quantization. For example, 4-bit weight quantization could maybe double the compression again, but is likely to need specialized hardware or some careful tuning to maintain accuracy. Very recent works (see MLSys 2024 reporting on Activation-Aware Quantization, for example) deliver a suggestion that even 3 to 4 bits might be well tolerable in producing good results for large models. If such techniques mature over time, we could incorporate them into our pipeline to push the edge deployment further, as possibly medium to large models like ResNet-50 running on micro devices.

Finally, but just as importantly, paradigm-shifting would be assessment under actual application conditions. Even though

our experiments were quite extensive, they were all still under control. For example, when thinking about a real-edge application like a security camera that performs detection, there are three things to consider: multi-tenant workloads (other, non-ML tasks running on that same device); the camera input pipeline; and thermal effects on the device when operating continuously. As a result, ongoing observation and model administration are required. Adding monitoring is obviously the goal of our containerized approach; we could add an agent to the container that could send performance metrics to a central server. This would enable A/B testing different model versions or quickly catch if the update degraded performance. We imagine that monitoring, updating, re-optimizing as data drifts, and so on will be critical to model lifecycle management in production edge deployments. Although beyond our current scope, our work contributes a major first step toward making the initial deployment efficient and robust.

In summary, it just emphasizes the multi-dimensional issues surrounding efficient model packaging and deployment. Our contributions lie at the juncture of model optimization and systems engineering, and these early positive results indicate that synergy across these two areas will be fundamental to unlocking full edge AI potential. Open challenges remain (automated, extreme compressing, dynamic edge-cloud partitioning), but there has been good progress so far: increasingly feasible to run sophisticated AI algorithms on ubiquitous edge-devices.

VI. CONCLUSION AND FUTURE WORK

Deploying AI models on edge hardware offers benefits that make systems responsive, private, and scalable. Their optimization is therefore pertinent: the proposed approach tackles the technological issues of joint packaging and inference at the edge. In this research work, model optimization is proposed under conditions that satisfy a selection of systems as a mutual link between design principles and application potentials to address MLOps and deep learning deployment dimensions.

Our experimental validation shows that our engineering can deliver highly efficient inference on edge devices. The results also show the value of the overall methodology: improvements in speed, memory, and energy usage, with negligible impact on accuracy. A correctly configured model, such as MobileNetV2, reaches ~ 100 ms on a Raspberry Pi machine limited by CPUs. The same model without quantization was too slow for real-time performance. This lowers the barrier for common applications—such as real-time image identification and sensor-data-driven decision making—that were once discouraged on edge devices for being too slow or power-hungry.

In the context of these experiments, a noteworthy observation is that a systemic approach yields maximum benefit from quantitative model compression and packaging. Compression is essential, and so is system-aware deployment. Although the technique described in this article encourages model developers to pursue end-to-end deployment strategies that meet latency and throughput specifications, additional flexibility for development will be a future venture through integration.

Notwithstanding, the scaffolding laid out by this document in terms of methodology and best practices will assist the AI community and industry practitioners working on edge AI applications. We aim to extend our methodology to more model classes, like transformers for NLP at the edge, and TinyML models for microcontrollers, as well as more use cases for edge-based AI applications. The implementation of automated optimization pipelines is on the roadmap. Dynamic deployments, in this sense, are promising extensions for the current study. In dynamic deployment, the system may decide whether to run a model locally or in a cloud environment based on contextual cues (such as available network conditions and required accuracy). This stands as an extension of our static deployment methodology. Such decision-making could enable systems that are both efficient and flexible.

In an age where IoT- and mobile-centered artificial intelligence are seeking optimal designs and applications, deploying advanced AI on the edge—where data is generated—creates new capabilities and a better user experience. This work contributes to that vision by providing an understanding of how to deploy complex algorithms under strict constraints. By maintaining a view of mixed-model compression, hardware acceleration, and deployment optimization, these findings support practical, edge-first AI solutions that can be widely distributed from cloud to edge.

REFERENCES

- [1] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. ICLR*, 2016.
- [2] B. Jacob, S. Kligys, B. Chen, *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. CVPR*, 2018, pp. 2704–2713.
- [3] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient convnets,” in *Proc. ICLR*, 2017.
- [4] A. G. Howard, M. Zhu, B. Chen, *et al.*, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv:1704.04861*, 2017.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. CVPR*, 2018, pp. 4510–4520.
- [6] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019, pp. 6105–6114.
- [7] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “XNOR-Net: ImageNet classification using binary convolutional neural networks,” in *Proc. ECCV*, 2016, pp. 525–542.
- [8] M. Courbariaux, Y. Bengio, and J.-P. David, “BinaryConnect: Training deep neural networks with binary weights during propagations,” in *NeurIPS*, 2015, pp. 3123–3131.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. CVPR*, 2016, pp. 770–778.
- [10] F. N. Iandola, S. Han, M. W. Moskewicz, *et al.*, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,” *arXiv:1602.07360*, 2016.
- [11] C. Banbury, V. J. Reddi, P. Torre, *et al.*, “MLPerf Tiny benchmark,” in *Proc. DAC*, 2021, pp. 1–6.
- [12] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, “HAQ: Hardware-aware automated quantization,” in *Proc. CVPR*, 2019, pp. 8612–8620.
- [13] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding sparse, trainable neural networks,” in *Proc. ICLR*, 2019.
- [14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. CVPR*, 2016, pp. 779–788.
- [15] P. Warden and D. Situnayake, “TensorFlow Lite Micro: Embedded machine learning on tiny systems,” *arXiv:2010.08678*, 2020.

- [16] NVIDIA Jetson community examples, “SSD-MobileNetV2 on Jetson Nano (COCO) real-time performance,” GitHub/Forums, accessed 2025. Available: <https://github.com/dusty-nv/jetson-inference>
- [17] NVIDIA Jetson community examples, “TensorRT-optimized SSD-MobileNetV2 (PETS, 37 classes) on Jetson Nano,” GitHub/Forums, accessed 2025. Available: <https://github.com/dusty-nv/jetson-inference>
- [18] Google Coral, “Edge TPU performance benchmarks (e.g., SSD-MobileNetV2 224x224),” accessed 2025. Available: <https://coral.ai/docs/edgetpu/benchmarks/>
- [19] T. Rice, “Real Time Inference on Raspberry Pi 4 (30 fps!),” PyTorch Tutorials, 2022. [Online]. Available: https://docs.pytorch.org/tutorials/intermediate/realtime_rpi.html
- [20] TensorFlow Lite, “Post-training quantization examples: MobileNetV2 model sizes (FP32/FP16/INT8),” accessed 2025. Available: https://www.tensorflow.org/lite/performance/post_training_quantization